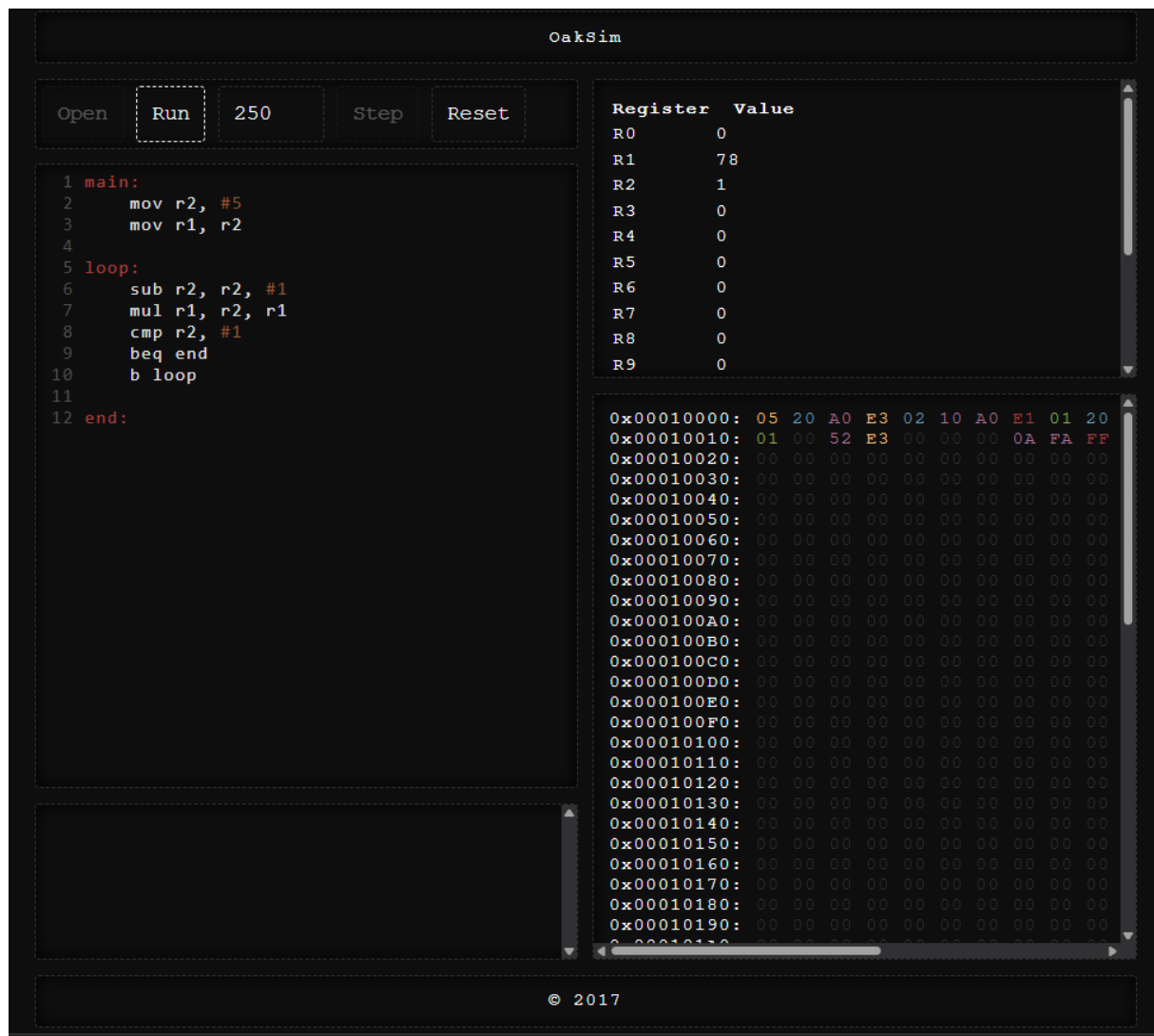


Template Week 4 – Software

Student number: 583259

Assignment 4.1: ARM assembly

Screenshot of working assembly code of factorial calculation:



Assignment 4.2: Programming languages

Take screenshots that the following commands work:

javac -version

```
erwina@erwina-VMware-Virtual-Platform:~$ javac -version
javac 21.0.9
erwina@erwina-VMware-Virtual-Platform:~$ java -version
openjdk version "21.0.9" 2025-10-21
OpenJDK Runtime Environment (build 21.0.9+10-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.9+10-Ubuntu-124.04, mixed mode, sharing)
```

java -version

```
erwina@erwina-VMware-Virtual-Platform:~$ javac -version
javac 21.0.9
erwina@erwina-VMware-Virtual-Platform:~$ java -version
openjdk version "21.0.9" 2025-10-21
OpenJDK Runtime Environment (build 21.0.9+10-Ubuntu-124.04)
OpenJDK 64-Bit Server VM (build 21.0.9+10-Ubuntu-124.04, mixed mode, sharing)
```

gcc -version

```
erwina@erwina-VMware-Virtual-Platform:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

python3 -version

```
erwina@erwina-VMware-Virtual-Platform:~$ python3 --version
Python 3.12.3
```

bash --version

```
erwina@erwina-VMware-Virtual-Platform:~$ bash -version
GNU bash, version 5.2.21(1)-release (x86_64-pc-linux-gnu)
Copyright (C) 2022 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>

This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

Assignment 4.3: Compile

Which of the above files need to be compiled before you can run them?

- **fib.c**
- **Fibonacci.java**

Which source code files are compiled into machine code and then directly executable by a processor?

- **fib.c** is compiled into machine code and becomes a directly executable program.

Which source code files are compiled to byte code?

- **Fibonacci.java** is compiled to bytecode and produces a **Fibonacci.class** file.

Which source code files are interpreted by an interpreter?

- **fib.py** and **fib.sh**.

These source code files will perform the same calculation after compilation/interpretation. Which one is expected to do the calculation the fastest?

- **fib.c** is expected to be the fastest because it is compiled into native machine code.

How do I run a Java program?

- `javac Fibonacci.java`
- `java Fibonacci`

How do I run a Python program?

- `python3 fib.py`

How do I run a C program?

- `gcc fib.c -o fib`
- `./fib`

How do I run a Bash script?

- `chmod +x fib.sh`
- `./fib.sh`

If I compile the above source code, will a new file be created? If so, which file?

- Yes.
- Compiling **fib.c** creates an executable file (usually **a.out** or **fib**).
- Compiling **Fibonacci.java** creates a **Fibonacci.class** file.

But **fib.py** and **fib.sh** do not produce new compiled output files.

Take relevant screenshots of the following commands:

- Compile the source files where necessary
- Make them executable
- Run them
- Which (compiled) source code file performs the calculation the fastest?

```
erwina@erwina-Virtual-Platform:~/Downloads/code$ cd ~/Downloads/code
erwina@erwina-Virtual-Platform:~/Downloads/code$ gcc fib.c -o fib
erwina@erwina-Virtual-Platform:~/Downloads/code$ javac Fibonacci.java

erwina@erwina-Virtual-Platform:~/Downloads/code$
erwina@erwina-Virtual-Platform:~/Downloads/code$ chmod +x fib.sh
erwina@erwina-Virtual-Platform:~/Downloads/code$
erwina@erwina-Virtual-Platform:~/Downloads/code$ ./fib
Fibonacci(18) = 2584
Execution time: 0.02 milliseconds
erwina@erwina-Virtual-Platform:~/Downloads/code$
erwina@erwina-Virtual-Platform:~/Downloads/code$ java Fibonacci

Fibonacci(18) = 2584
Execution time: 0.37 milliseconds
erwina@erwina-Virtual-Platform:~/Downloads/code$
erwina@erwina-Virtual-Platform:~/Downloads/code$ python3 fib.py

Fibonacci(18) = 2584
Execution time: 0.41 milliseconds
erwina@erwina-Virtual-Platform:~/Downloads/code$
erwina@erwina-Virtual-Platform:~/Downloads/code$ ./fib.sh
Fibonacci(18) = 2584
Execution time 6473 milliseconds
erwina@erwina-Virtual-Platform:~/Downloads/code$
erwina@erwina-Virtual-Platform:~/Downloads/code$
```

Assignment 4.4: Optimize

Take relevant screenshots of the following commands:

- a) Figure out which parameters you need to pass to **the gcc** compiler so that the compiler performs a number of optimizations that will ensure that the compiled source code will run faster. **Tip!** The parameters are usually a letter followed by a number. Also read **page 191** of your book, but find a better optimization in the man pages. Please note that Linux is case sensitive.
 - GCC provides optimization options such as `-O2` and `-O3` that can be used when compiling C programs. These options tell the compiler to optimize the code so it can run faster. The `-O3` option applies more optimization than `-O2` and is commonly used when performance is important.

```
compilation terminated.
erwina@erwina-VMware-Virtual-Platform:~$ cd Downloads/code
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ gcc fib.c -o fib_no_opt
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ gcc -O3 fib.c -o fib_opt
gcc: error: unrecognized command-line option '-O3'
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ gcc -O3 fib.c -o fib_opt
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ ./fib_no_opt
Fibonacci(18) = 2584
Execution time: 0.04 milliseconds
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ ./fib_opt
Fibonacci(18) = 2584
Execution time: 0.01 milliseconds
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$
```

- b) Compile **fib.c** again with the optimization parameters
 - To compile `fib.c` with optimization enabled, I used the gcc optimization parameter `-O3`, which optimizes the code to improve execution speed.

```
erwina@erwina-VMware-Virtual-Platform:~$
erwina@erwina-VMware-Virtual-Platform:~$ cd ~/Downloads/code
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ gcc -O3 fib.c -o fib_opt
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$
```

- c) Run the newly compiled program. Is it true that it now performs the calculation faster?
- Yes, the optimized program runs faster. After running `./fib_opt`, the execution time decreased compared to the original `./fib` program. This confirms that GCC optimization parameters (such as `-O3`) improve runtime performance.

```
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ ./fib_no_opt
Fibonacci(18) = 2584
Execution time: 0.05 milliseconds
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ ./fib_opt
Fibonacci(18) = 2584
Execution time: 0.01 milliseconds
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$
```

- d) Edit the file `runall.sh`, so you can perform all four calculations in a row using this Bash script. So the (compiled/interpreted) C, Java, Python and Bash versions of Fibonacci one after the other.
- I edited `runall.sh` so that it runs the C, Java, Python and Bash versions of the Fibonacci program one after the other.

```
#!/bin/bash

echo "Running optimized C version:"
./fib_opt
echo

echo "Running Java version:"
java Fibonacci
echo

echo "Running Python version:"
python3 fib.py
echo

echo "Running Bash version:"
./fib.sh
echo

Erwina Rompas|
```

- When running the script, the programs C, Java, Python and Bash are executed in sequence. The output shows that all versions produce the same result, but the execution time differs between the programming languages.

```
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ ./runall.sh
Running optimized C version:
Fibonacci(18) = 2584
Execution time: 0.01 milliseconds

Running Java version:
Fibonacci(18) = 2584
Execution time: 0.24 milliseconds

Running Python version:
Fibonacci(18) = 2584
Execution time: 0.45 milliseconds

Running Bash version:
Fibonacci(18) = 2584
Execution time 6396 milliseconds
```

Assignment 4.5: More ARM Assembly

Like the factorial example, you can also implement the calculation of a power of 2 in assembly. For example you want to calculate $2^4 = 16$. Use iteration to calculate the result. Store the result in r0.

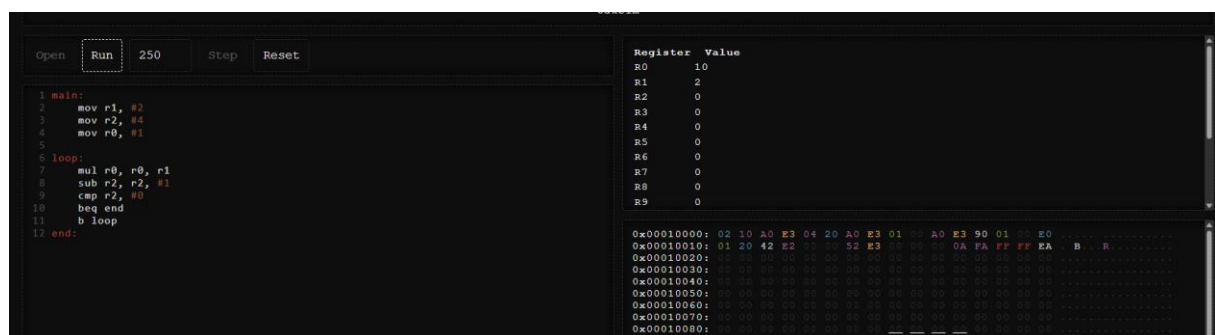
Main:

```
mov r1, #2
```

```
mov r2, #4
```

Loop:

End:

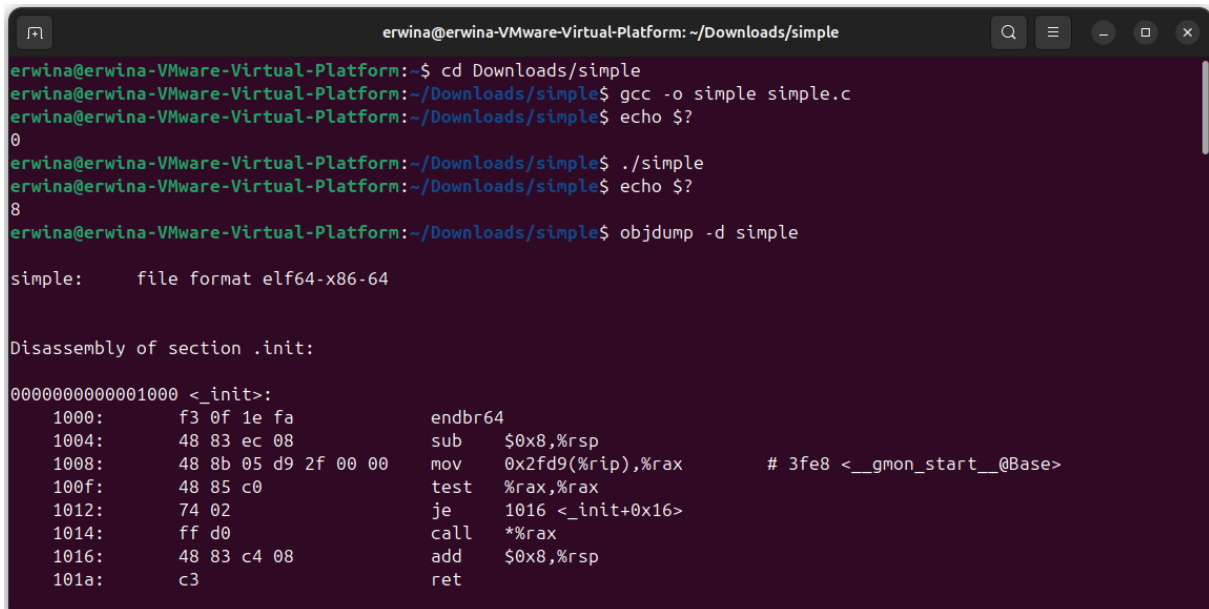


In this assignment, the assembly program is executed step by step. The registers and memory view show how values change while the program runs. This makes it possible to see how instructions are executed and how the loop works.

Complete the code. See the PowerPoint slides of week 4.

Screenshot of the completed code here.

simple.c



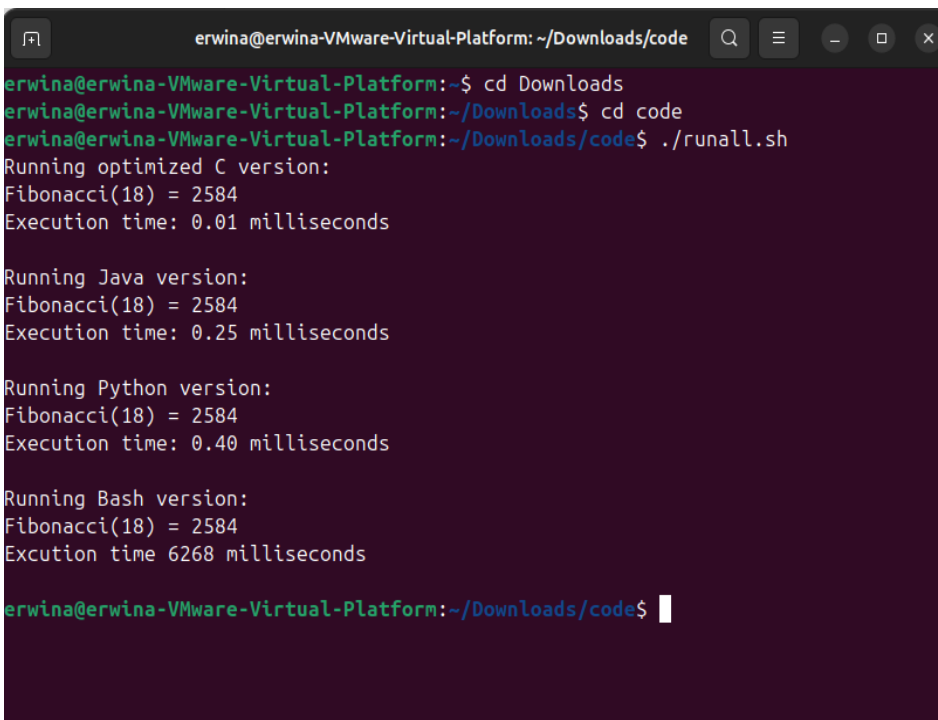
```
erwina@erwina-VMware-Virtual-Platform: ~/Downloads/simple
erwina@erwina-VMware-Virtual-Platform:~$ cd Downloads/simple
erwina@erwina-VMware-Virtual-Platform:~/Downloads/simple$ gcc -o simple simple.c
erwina@erwina-VMware-Virtual-Platform:~/Downloads/simple$ echo $?
0
erwina@erwina-VMware-Virtual-Platform:~/Downloads/simple$ ./simple
8
erwina@erwina-VMware-Virtual-Platform:~/Downloads/simple$ echo $?
8
erwina@erwina-VMware-Virtual-Platform:~/Downloads/simple$ objdump -d simple

simple:      file format elf64-x86-64

Disassembly of section .init:

0000000000001000 <_init>:
1000:      f3 0f 1e fa      endbr64
1004:      48 83 ec 08      sub    $0x8,%rsp
1008:      48 8b 05 d9 2f 00 00  mov    0x2fd9(%rip),%rax        # 3fe8 <__gmon_start__@Base>
100f:      48 85 c0      test   %rax,%rax
1012:      74 02      je     1016 <_init+0x16>
1014:      ff d0      call   *%rax
1016:      48 83 c4 08      add    $0x8,%rsp
101a:      c3      ret
```

code.zip



```
erwina@erwina-VMware-Virtual-Platform: ~/Downloads/code
erwina@erwina-VMware-Virtual-Platform:~$ cd Downloads
erwina@erwina-VMware-Virtual-Platform:~/Downloads$ cd code
erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$ ./runall.sh
Running optimized C version:
Fibonacci(18) = 2584
Execution time: 0.01 milliseconds

Running Java version:
Fibonacci(18) = 2584
Execution time: 0.25 milliseconds

Running Python version:
Fibonacci(18) = 2584
Execution time: 0.40 milliseconds

Running Bash version:
Fibonacci(18) = 2584
Execution time 6268 milliseconds

erwina@erwina-VMware-Virtual-Platform:~/Downloads/code$
```

hello.zip on Ubuntu


```
erwina@erwina-VMware-Virtual-Platform:~$ cd ~/Downloads/hello
erwina@erwina-VMware-Virtual-Platform:~/Downloads/hello$ javac Hello.java
erwina@erwina-VMware-Virtual-Platform:~/Downloads/hello$ gcc hello.c -o hello
erwina@erwina-VMware-Virtual-Platform:~/Downloads/hello$
erwina@erwina-VMware-Virtual-Platform:~/Downloads/hello$ java Hello
Hello world!
erwina@erwina-VMware-Virtual-Platform:~/Downloads/hello$
erwina@erwina-VMware-Virtual-Platform:~/Downloads/hello$ ./hello
Hello, World!
erwina@erwina-VMware-Virtual-Platform:~/Downloads/hello$
erwina@erwina-VMware-Virtual-Platform:~/Downloads/hello$
```

hello.zip on Windows (Host PC)

```
C:\Users\erwin>cd C:\Users\erwin\Desktop

C:\Users\erwin\Desktop>cd C:\Users\erwin\Desktop\hello

C:\Users\erwin\Desktop\hello>java Hello
Hello world!

C:\Users\erwin\Desktop\hello>
```

After moving both compiled programs from the Ubuntu VM to the Windows host system, only the Java program can be executed on Windows. The C program cannot be run because it was compiled for Linux and is not compatible with Windows.